

| 2026.02.14 - Lecture

| Method References

Method References are a *shorthand syntax* that replaces the `lambda expression`

- use the `::` operator without any parameters
- can be used to make simple *lambda expressions*

Essentially, by using method references you're telling Java "*I don't want to write out a whole new block of code here. Just **point directly** to this existing method*"

To make this work, Java uses **Functional Interfaces** (`BiFunction`, `Supplier` and `Runnable` → think of them as `special variable types` that are designed to **hold a method** vs. holding data.

| Four Types of Method References



Method References Examples

```
1  class MathUtils
2  {
3      static int add(int a, int b)
4      {
5          return a + b;
6      }
7  }
8
9  class Printer
10 {
11     void printMessage()
12     {
13         System.out.println("Hello!");
14     }
15 }
16
17 class Car
18 {
19     Car()
20     {
21         System.out.println("Car created.");
22     }
23 }
24
```

| `static` method



static method reference

```

1 public class Main {
2     public static void main(final String[] args)
3     {
4         // Static Method Reference
5         BiFunction<Integer, Integer, Integer> adder = MathUtils::add;
6
7         System.out.println(adder.apply(3, 4)); // 7
8     }
9 }

```

- `ClassName::staticMethod` → `MathUtils::add`

You have a `static` method that adds two numbers.

- You are **storing a reference** to that method inside a `BiFunction` variable named `adder`.
- When you call `adder.apply(3, 4)` $\iff$ `{java}MathUtils.add(3, 4)`

What is a `BiFunction`?

A **functional interface** (`BiFunction<Input1, Input2, Output>`) is a **built-in** Java type that **takes in** `two inputs` → **returns** `one output`.

Instance method of an object of a certain type

```

instance method reference → obj TYPE

1 public class Main
2 {
3     // Instance Method Reference (any obj of a TYPE)
4     public static void main(final String[] args)
5     {
6         BiFunction<String, String, Integer> comparator = String::compareTo;
7
8         System.out.println(comparator.apply("apple", "banana")); // -value (-1)
9     }
10 }

```

- `ClassName::instanceMethod` → `String::compareTo`

You are referencing an **instance method**, but *aren't attaching it* to a specific string yet.

- You are telling Java the **blueprint** `compareTo` for *what to do* when it **eventually receives** the objects (`{java}comparator.apply("apple", "banana");`)
- When you call `comparator.apply("apple", "banana")`, Java takes the **first parameter** (`apple`) and makes it the **target object**, and passes the **second parameter** (`into the method`).
- `comparator.apply("apple", "banana")` $\iff$ `"apple".compareTo("banana");`

Instance method of an existing object



instance method reference → SPECIFIC obj

```

1 public class Main
2 {
3     public static void main(final String[] args)
4     {
5         // Instance Method Reference (SPECIFIC object)
6         Printer printer = new Printer();
7         Runnable<Void> message = printer::printMessage;
8
9         message.run();
10    }
11 }

```

- `object::instanceMethod` → `printer::printMessage`

You **already instantiated** a **specific object** `Printer printer = new Printer();`

- You want to **save a reference** to that **exact object's method** to run later.
- Taking a **specific object** and typing its **method** to a **variable**.
- `message.run()` $\iff$ `printer.printMessage();`

| constructor method



constructor reference

```

1 public class Main
2 {
3     public static void main(final String[] args)
4     {
5         Supplier<Car> carSupplier = Car::new;
6
7         carSupplier.get();
8     }
9 }

```

- `ClassName::new` → `Car::new`

Instead of referencing a **method**, you are referencing a **class constructor**.

- You create a `carSupplier = Car::new` and treat this **variable** like a **factory**.
- Every single time you call `carSupplier.get()`, it executes `new Car()` and hands you a **new** `Car` object.
- `carSupplier.get()` $\iff$ `new Car()`

| Lambda vs. method reference

`(args)->Class.staticMethod(args)` ← can be **replaced** by → `Class::staticMethod` *any arguments will be passed automatically by Java.*

| Generics

Generics allow us to *write methods, classes, interfaces, and collections* that enforce type safety at **compile-time**.

| Advantages of Generics

1. No Casting Required

- `Box<String>` knows that it must store and return `String` objects.
- `Box<Integer>` knows that it must store and return `Integer` objects.

2. Type-Safe (reduced runtime error)

- Generics provide **compile-time** type checking.
- If you try to store a `String` in a `Box<Integer>`, the compiler will *throw an error*, preventing that mistake.

3. Reusability

- Use the same `Box<T>` class for any type (`String`, `Integer`, `CustomObjects`, etc ...)
 - reducing code duplication and making classes more versatile



Without Generics

```
1 public class BoxWithoutGenerics
2 {
3     private Object t;
4
5     public void set (final Object t)
6     {
7         this.t = t;
8     }
9
10    public Object get()
11    {
12        return t;
13    }
14 }
15
16 public class Main
17 {
18     public static void main(final String[] args)
19     {
20         final BoxWithoutGenerics box = new BoxWithoutGenerics();
21
22         box.set("Hello"); // storing a string
23
24         final String s;
25         s = (String) box.get(); // EXPLICIT CASTING IS NEEDED
26
27         box.set(123); // storing an integer
28         final Integer i;
29         i = (Integer) box.get(); // EXPLICIT CASTING
30     }
31 }
```

- Trying to assign `box.get();` to a variable without an explicit cast returns a **compiler error**: `java: incompatible types: java.lang.Object cannot be converted to java.lang.String`



With Generics (Box<T>)

```
1 public class BoxWithGenerics<T>
2 {
3     private T t;
4
5     public void set(final T t)
6     {
7         this.t = t;
8     }
9
10    public T get()
11    {
12        return t;
13    }
14
15    public static void main(String[] args)
16    {
17        final BoxWithGenerics<String> stringBox = new BoxWithGenerics<>();
18        final BoxWithGenerics<Integer> intBox = new BoxWithGenerics<>();
19
20        stringBox.set("Hello"); // storing a string
21
22        final String s;
23        s = stringBox.get(); // NO CASTING NEEDED
24        System.out.println(s);
25
26
27
28        intBox.set(123); // storing and integer
29
30        final Integer i;
31        i = intBox.get(); // NO CASTING NEEDED
32        System.out.println(i);
33    }
34 }
```

Generic Types

Common Conventions

But hypothetically, can use any names we'd like.

1. **T** and **U** → "*type*" parameters; can represent any reference type
 - often used for types
2. **E** → for *elements*
3. **K** and **V** → *key* and *value*

Generic Methods

Same core idea as Generic Classes (like `ArrayList<String>`), but applied to a **single method** instead of a whole class.

Generic methods introduce their own *type parameters*, independent of the class type parameters.

- allows for **flexibility** when performing actions that don't depend on the class's generic type.

```
1 public class Utility
2 {
3     public static <T extends Comparable<T>> T getMax(T a, T b)
4     {
5         return a.compareTo(b) > 0 ? a : b;
6     }
7 }
```

- `public static` → normal method modifiers
- `<T extends Comparable<T>>` → the magic switch
 - using a placeholder `T`, whatever `T` is, it **MUST implement** the `Comparable` interface so I can sort it.
- `T` → the return type; the method will return whatever type `T` ends up being (pass in apples? you get back apples)
- `getMax(T a, T b);` → the method takes two inputs, both of type `T`
- `return a.compareTo(b) > 0 ? a : b;` → the logic.
 - `T` is guaranteed `Comparable`, Java knows that `a` has a `.compareTo()` method, so the code is safe to run.

🤖 Analogy: The Universal Sorting Machine

Imagine you work in a factory with a magic sorting machine

The Problem: People keep bringing different things to sort.

- Sometimes it's a pile of Apples (you want the heaviest).
- Sometimes it's a stack of Books (you want the thickest).
- Sometimes it's a group of Students (you want the highest grades).

The Bad Way (Overloading):

- You build **three** *completely separate* sorting machines:
 - `AppleSorter`
 - `BookSorter`
 - `StudentSorter`
- Every time a new item arrives, you have to build a new machine.

The Good Way (Generic Method)

- You build *one magic machine*. Before you turn it on, you flip a switch and say ... "For this run, `<T>`"
 - `Sorter`

Example Use Case

Find the largest of two items.



The 'Bad' Way

```
1 public static Integer getMaxInt(Integer a, Integer b)
2 {
3     return a.compareTo(b) > 0 ? a : b;
4 }
5
6 public static String getMaxString(String a, String b)
7 {
8     return a.compareTo(b) > 0 ? a : b;
9 }
```



The 'Good' Way

```
1 public class Utility
2 {
3     public static <T extends Comparable<T>> T getMax(T a, T b)
4     {
5         return a.compareTo(b) > 0 ? a : b;
6     }
7
8     public static void main(final String[] args)
9     {
10         Integer biggestInt = Utility.getMax(5, 10);
11         System.out.println(biggestInt); // 10
12
13         String biggestWord = Utility.getMax("Apple", "Zebra");
14         System.out.println(biggestWord); // Zebra
15     }
16 }
```

Best Use Case

You can **write utility methods** (sorting, searching or comparing) that *works universally for any class*, as long as that class `follows the rules` you set (like implementing `Comparable`)

- Don't have to make the entire `Utility` class generic; just that **specific method has its own independent type parameters**.

Bounded Types in Generics

Putting a **restriction/limit** on what `T` is allowed to be.

ALWAYS USE `extends` **to set a bound**

You can use **bounds** to **restrict generic types** with the `extends` or `super` keywords. This ensures that only certain specific types can be passed while still allowing flexibility.



Bounded-Type Example

```
1 public <T extends Number> void addNumbers(T a, T b)
2 {
3     System.out.println(a.doubleValue() + b.doubleValue());
4 }
```

In this example, the method accepts only types that `extends Number`, ensuring **type-safety** with **numeric types**.

- allows you to pass `Integer`, `Double` or `Float`
- allows you to safely type `a.doubleValue() + b.doubleValue();`
 - without this **bound**, Java would treat `a` and `b` as **plain** `Object`s and will not allow you to do math with them.

Comparing with `<T extends Comparable> ...`

- This rule says: "you can pass any object as long as **it can be compared/sorted**" ← allows you to pass in Strings, Dates, or Integers.
- allows us to safely type `a.compareTo(b)`

Bounded Types with Classes

This generic typing allows `T` and `U` to be of absolutely **any reference type** at all.

- Instead, we can put **upper and/or lower bounds** on the allowed types.



Bound Types with Classes

```
1 class Pair<T, U>
2 {
3     private final T first;
4     private final U second;
5
6     public Pair(final T first,
7                 final U second)
8     {
9         this.first = first;
10        this.second = second;
11    }
12
13    public T getFirst()
14    {
```



```

15         return first;
16     }
17
18     public U getSecond()
19     {
20         return second;
21     }
22 }

```

Upper Bound

- restricts a **type of generic parameter** to a **specific type** or its **subclass** using **extends** keyword.
- allows for **type-safety**, ensuring that only *certain types* are used, **guaranteeing access to specific methods**, we can **use methods available in upper-bound class**.

Lower Bound

- restricts a **wildcard** to a **specific type** of its **superclass** using the **super** keyword

Wildcards in Generics



Wildcards in Generics

```

1 public void printList(final List<?> list)
2 {
3     for (final Object elem : list)
4     {
5         System.out.println(elem);
6     }
7 }

```

Wildcards prove flexibility in generics, **allowing a method to accept** a *range of types*.

Three Common Types of Wildcards

- ?** → unbounded wildcard
- ? extends T** → upper-bounded wildcard (**T** or any *subclass*)
- ? super T** → lower-bounded wildcard (**T** or any *superclass*)

Unbounded Wildcard **?**

Only allows **reading**; can look, count, put it back.

- CANNOT write** (add) into it

```

1 public void printList(List<?> list)
2 {
3     for (final Object elem : list)

```

```

4      {
5          System.out.println(elem);
6      }
7  }

```

⚠️ List<?> vs. List<Object>

- A `List<?>` means "A list of *some unknown specific type*". It can accept a `List<String>`.
- A `List<Object>` means "A list *specifically designed to hold mixed Objects*".
 - It **CANNOT** accept a `List<String>`

Java considers `List<Object>` and `List<String>` to be **COMPLETELY UNRELATED TYPES**.

A `String` is an `Object`, but in Java Generics, a `List<String>` is **NOT** a subclass of `List<Object>`, hence → `List<?>` is read-only.

Multiple Bounds in Generics

Stacking **multiple bounds** (rules) together.

Multiple bounds allow you to specify that a type must satisfy multiple conditions.



Multiple Bounds in Generics

```

1  public <T extends Comparable<T> & Serializable> T getMax(T a, T b)
2  {
3      return a.compareTo(b) > 0 ? a : b;
4  }

```

- `T` **must** be both `Comparable` and `Serializable`.

Analogy

- **No Bounds** `<T>`: "I will hire literally anyone."
- **Single Bound** `<T extends Programmer>`: "I will hire anyone, as long as they are a `Programmer`".
- **Multiple Bounds** `<T extends Programmer & Bilingual>`: "I will hire anyone who is a `Programmer` **AND** they must be `Bilingual`".

RULES

Syntax: `<T extends Bound1 & Bound2 & Bound3>`

1. **First Bound** (`Bound1`): **MUST** be a `class` or an `interface`
2. **Subsequent Bounds** (`Bound2`, `Bound3`, etc.): any bound listed *after the first one* **MUST** be an `interface`

Remember Java rules: you can only **extend one** class, but you can **implement as many** interfaces as you want.

| Generics and Inheritance

Generics and inheritance interact in specific ways:

- A `List<Dog>` is **NOT a subtype** of `List<Animal>` → even if `Dog` is a subtype of `Animal`.

| Covariance and Contravariance

| Questions

-

| To Review

-

| Linked Notes

- [COMP2522.M04](#)